An Analysis of Monte Carlo Simulations for Forecasting Software Projects

Pedro Miranda
up201506574@fe.up.pt
Faculty of Engineering,
University of Porto
Porto, Portugal

J. Pascoal Faria
jpf@fe.up.pt
Faculty of Engineering,
University of Porto
INESC TEC
Porto, Portugal

Filipe F. Correia
filipe.correia@fe.up.pt
Faculty of Engineering,
University of Porto
INESC TEC
Porto, Portugal

Ahmed Fares
ahmed.fares@fe.up.pt
Faculty of Engineering,
University of Porto
INESC TEC
Porto, Portugal

Ricardo Graça
ricardo.graca@fraunhofer.pt
Fraunhofer Portugal AICOS
Porto, Portugal

João Mendes Moreira
jmoreira@fe.up.pt
Faculty of Engineering,
University of Porto
INESC TEC
Porto, Portugal

ABSTRACT

Forecasts of the *effort* or *delivery date* can play an important role in managing software projects, but the estimates provided by development teams are often inaccurate and time-consuming to produce. This is not surprising given the uncertainty that underlies this activity. This work studies the use of Monte Carlo simulations for generating forecasts based on project historical data. We have designed and run experiments comparing these forecasts against what happened in practice and to estimates provided by developers, when available. Comparisons were made based on the mean magnitude of relative error (MMRE). We did also analyze how the forecasting accuracy varies with the amount of work to be forecasted and the amount of historical data used. To minimize the requirements on input data, delivery date forecasts for a set of user stories were computed based on takt time of past stories (time elapsed between the completion of consecutive stories); effort forecasts were computed based on full-time equivalent (FTE) hours allocated to the implementation of past stories. The MMRE of delivery date forecasting was 32% in a set of 10 runs (for different projects) of Monte Carlo simulation based on takt time. The MMRE of effort forecasting was 20% in a set of 5 runs of Monte Carlo simulation based on FTE allocation, much smaller than the MMRE of 134% of developers' estimates. A better forecasting accuracy was obtained when the number of historical data points was 20 or higher. These results suggest that Monte Carlo simulations may be used in practice for delivery date and effort forecasting in agile projects, after a few initial sprints.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SAC '21, March 22–26, 2021, Virtual Event, Republic of Korea
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8104-8/21/03…$15.00
https://doi.org/10.1145/3412841.3442030

CCS CONCEPTS

• **Software and its engineering** → *Software development process management*;

KEYWORDS

Forecasting, Software Development, Monte Carlo Simulation, Software Delivery, Developer Effort

ACM Reference Format:
Pedro Miranda, J. Pascoal Faria, Filipe F. Correia, Ahmed Fares, Ricardo Graça, and João Mendes Moreira. 2021. An Analysis of Monte Carlo Simulations for, Forecasting Software Projects. In *The 36th ACM/SIGAPP Symposium on Applied Computing (SAC '21), March 22–26, 2021, Virtual Event, Republic of Korea.* ACM, New York, NY, USA, Article 4, 9 pages. https://doi.org/10.1145/3412841.3442030

1 INTRODUCTION

Software developers are usually not able to estimate accurately the effort, time, and cost of a project to be developed. This is inherited to the uncertainty that underlies their activity [23]. Delays in relation to expectations affect the development team but can also impact other parts of the company, such as staffing or marketing. This may lead to inefficiencies or even to losing the trust of stakeholders.

Previous research and reports from practitioners show encouraging results when using Monte Carlo simulations for forecasting software projects [27][10]. However, the technique is not widely used by the software industry [17]. Additionally, academic works that try to study its applicability are hard to find, and there is a thin practical evidence of the accuracy of these methods for forecasting delivery and effort of software projects.

Therefore, we seek to study if a data-driven method such as Monte Carlo is well suited for forecasting software projects. Specifically, the objectives and contributions of this work are:

- To study the feasibility of using the Monte Carlo method to achieve accurate forecasts, that is, forecast with smaller relative errors, and with a low effort for software developers.

- Understand how the accurateness of the Monte Carlo method varies with the amount of historical data used and with the amount of work to be forecasted.

The structure of this paper is divided into 5 more sections.

Section 2 reviews methods for software development estimation and forecasting, positioning the Monte Carlo Method.

In Section 3, two different delivery forecasting approaches are discussed, *takt time* and *effort*, as well as the Monte Carlo core algorithm.

In Section 4, the outputs produced by both approaches that were implemented with the Monte Carlo method are discussed and represented.

In Section 5, the experiments for each delivery forecasting approach are reported, their results are discussed, and some findings of their effectiveness and accuracy are taken.

In Section 6, conclusions are drawn, problems that arise during the development are mentioned, and possible future work is discussed.

2 RELATED WORK

This section seeks to review key notions around forecasting and estimation methods, and to support understanding what options are available when predicting delivery or effort for software projects.

We adopt the following definitions by Keogh [14]:

- **Estimate**, a single value prediction of what is the real effort.
- **Forecast**, a probabilistic prediction of a range of values or an exact value with confidence levels associated. Forecasts can be classified as:
  - Point Forecasts, single value exact forecasts.
  - Interval Forecasts, range of values forecasts with a confidence inherent.

Keogh defends that forecasting, for its probabilistic nature, will often provide more accurate and reliable predictions than estimates.

In *forecasting* and *estimation* of software development, several metrics may be considered to classify inputs and outputs. Some of these metrics are the *understandability*, *representability*, and *difficulty to calculate*.

Two important metrics of software development that will be important for the rest of the document are *Takt Time* and *Throughput*.

Takt Time: *Takt* is a German word for a rhythm or beat. Takt Time describes the regularity of that beat. In production line manufacturing, Takt Time is the time period at which each manufactured item is finished. Using Takt Time, a manufacturer can run the line at a pace that is in correspondence with demand. It also provides an easy way to work out how long it would take to produce large batches of items [10].

Throughput: Throughput is the number of items completed in a given time window or iteration. It is a measure of output, how much is being done. Measuring daily throughput as well as trends over time provides insight into both the rate and rhythm of delivery [6].

The main metrics that we use for error quantification are the Relative Error (RE) or Percentage Error (PE) with the Equation (1), the Magnitude of Relative Error(MRE) with the Equation (2) and the Mean Magnitude of Relative Error (MMRE) with the Equation (3)

$$RE = \frac{\text{Actual value - Estimated value}}{\text{Actual value}} \tag{1}$$

$$MRE = |RE| \tag{2}$$

$$MMRE = \frac{\sum_{i=1}^{n} MRE_i}{n} \tag{3}$$

These metrics will be used to assess the Monte Carlo method, as will be discussed in Section 5.

With these notions reviewed, we can now analyze the different types of estimating and forecasting methods. The methods can be divided into 4 main types [2]:

- **Empirical Methods**, methods that gather information by means of direct observation and experimentation.
- **Theory-Based Methods**, methods based on an explicit theory or logic model.
- **Regression Methods**, statistical methods for calculating the relationships between a dependent variable and one or many independent variables.
- **Machine Learning Methods**, methods that work with machine learning and focus on the learning ability inherent.

Table 1 shows the analysed methods, characterized by *type* of method, *ease to collect the input* required (easy,medium,hard), *accuracy of the output* as described in other works (unsatisfactory, satisfactory, good), *understandability of the method and its procedures* (easy, medium, hard) and the *applicability in agile context* (Yes, No), that is, if it is possible to apply the method to an iterative style of work (e.g., using *sprints*).

COCOMO is a theory-based method that, in synthesis, takes substantial inputs when using the intermediate or Detailed types while the Basic type takes fewer inputs but provides low accuracy [26]. Most of these inputs are expert-based, meaning that an expert must estimate some inputs to make the forecast. This could lead to subjectivity and bias. Some benefits are the fact that it is simple to implement and it is also relatively simple to understand. A possible implementation that mitigates some of the problems of COCOMO and increases its accuracy is combining it with fuzzy logic [26]. It is not easily applied in agile contexts, since it does not analyze past iterations of a project. In terms of accuracy, it can be said that the results produced are not very accurate [25].

SLIM is a method inspire by the Putnam model. SLIM's major limitation is the fact that the method in an experiment of 750 projects only worked properly in 251 [9], which represents a high number of projects where the model did not work properly. Also, it is a target of criticism because of its assumptions, like the disregard of the initial stages of a project [26]. Compared to the other methods presented, SLIM does not add anything that could be considered as a benefit in relation to the others. The method is difficult to apply in agile contexts since it does not analyze past sprints of a given project. However, it provides a satisfactory accuracy and is relatively easy to understand.

Both **Wideband Delphi** [24] and **Planning Poker** [11] are expert-opinion methods and both have proven to achieve a satisfactory accuracy without decreasing the interpretability of the results due to its simplicity. Their main liabilities are the fact that it requires longer meetings to make the estimations and is also subject

Method	Type	Input Collection	Output Accuracy	Method Understandability	Applicability in Agile
Cocomo	Theory Based	Medium	Unsatisfactory [25]	Easy	No
SLIM	Theory Based	Hard	Satisfactory [25]	Hard	No
Wideband Delphi	Empirical	Easy	Satisfactory [6]	Easy	Yes
Planning Poker	Empirical	Easy	Satisfactory [6]	Easy	Yes
Angel	Empirical	Medium	Unsatisfactory [20]	Easy	No
Monte Carlo	Theory Based	Easy	Good [16]	Easy	Yes
SEER-SEM	Regression	Hard	Satisfactory [25]	Hard	Yes
Support Vector Machine	Machine Learning	Easy	Good [15]	Hard	Yes
Bayesian Network	Machine Learning	Medium	Good [1]	Hard	Yes

Table 1: Synthesis of methods

to human bias. A great benefit both Planning Poker and Wideband Delphi have, in relation to the others, is the fact that development teams are more comfortable using this expert-opinion method because of the discussion between different perspectives and the great communication it provides. Among the expert-opinion methods, planning poker is simpler and with a more focused discussion achieves better results than Wideband Delphi. Both methods can be used in agile context since new meetings can be done for estimating at the end of every iteration.

Angel [20], since it is an analogy method, proves itself to be easy to understand and to interpret. The main limitations are the inability to handle missing data and non-exact values and the fact that the premise of this type of method is partly wrong. These limitations make it produce not very accurate estimations. When combining methods like Angel with Fuzzy logic [12], we saw that these limitations are mitigated and better accuracy is achieved. The method is not easily implemented in agile contexts since it doesn't analyze past iterations of the same project.

Monte Carlo proves to be a method with good accuracy [27] and that is easy to implement and to understand, with some literature even giving simple examples of the process of this method [4]. Another great benefit is the granularity inherent, with an easy interpretation of the degree each input affects the final forecast [27]. The method is easily implemented in agile contexts since it can be implemented to analyze past iterations of the project. Magennis suggests that the method produces accurate forecasts [16].

SEER-SEM, although showing satisfactory accuracy, presents itself as a hard method to understand and interpret, partially because of its numerous inputs (could go over the 50 inputs) [13] and because of its specific details that make really hard the interpretation of the relation between inputs and outputs [22]. When used in combination with Fuzzy logic, this difficult understanding is mitigated and the accuracy even improves, in relation to SEER-SEM alone [8]. The method can be adopted when using agile methodologies.

Both **SVM** [7] and **BN** [1] are presented as relatively difficult to understand, with some studies even calling the SVM method a black box one [5]. However, these methods show great accuracy [19], the ability to handle missing data, the possibility of discovering existing patterns between projects, the fact of avoiding overfitting to data, and the capability of generalization [18], not being confined to one type of projects, what translates to these methods producing accurate results. Both methods can work in agile contexts since both can analyze past iterations of a project.

We find that the most promising methods are the machine learning methods and the Monte Carlo method. They share some characteristics, namely, the ease to collect the inputs required, the good accuracy of the output, and the applicability to agile contexts. Being data-driven methods, makes them immune to human biases, and when used to forecast the delivery of a set of user-stories, they are robust to a possibly high-variance of the history data. The Monte Carlo method has the added benefit of having good interpretability, which machine-learning methods don't always have [3].

This overview of related work provides the context to the study that is reported in this paper, where we seek to understand how data-driven forecasting methods (and, in particular, the Monte Carlo method) are able to provide good levels of accuracy without implying an added effort for the software development team.

3 FORECASTING WITH MONTE CARLO

The goal of this work is to explore the extent to which the Monte Carlo method is able to provide lean and accurate forecasts. With this study, we seek to bring new insights about the applicability of this method to software development, as it has not yet been the focus of significant study by researchers and practitioners [10][27], so there is still a lack of evidence of its performance and of under which conditions it is most effective.

3.1 Monte Carlo Model

In order to implement the core algorithm of the Monte Carlo method, we have defined a function that receives the historical data and the number of tasks/user-stories to forecast and that returns the median, mean, optimist, and pessimist forecast, that works as the confidence interval of the forecast.

Given a set of historical values, our implementation of the Monte Carlo method selects a few random values from that historical data and adds them to simulate the value for all the user-stories to forecast. This set of values are then sorted and we exclude the smaller and higher values, according to a confidence interval (in the following experimentation, we use a confidence interval of 95%, i.e. include 95% of the values produced by the model). The method then returns the mean, median, optimist (lower), and pessimist (upper) forecasts for that number of user-stories. The median is effectively our forecast, and the confidence interval is defined by our optimist and pessimist values.

The inputs required by this algorithm are the historical dataset, the confidence interval and the number of user-stories to forecast.

3.2 Takt Time Forecasting

When using takt time, the date of the final delivery D, that is provided as output, is calculated by summing the values of the M forecasts, and using as input the N takt times t_i, with $i = 1, 2, 3, ..., N$. The values of t_i are the time periods between the completions of historical user-stories c_n, with $n = 1, 2, 3, ..., N + 1$. In this way we extract the periods of time between the finishing dates of a set of user-stories, that is, the amount of time no user-story was completed, to find the *rate* of delivery. With these takt times, we can forecast how long will a user-story or a set of user-stories take to be completed.

3.3 Effort Forecasting

Effort is another means to forecast a user-story or a set of user-stories. This approach takes as input the amount of time the developer assigned to the user-story was really working on that user-story.[1] For a set of N historical user-stories, the effort of each user-story fe_j, with $j = 1, 2, 3, ..., N$ is calculated as the sum of eph_i, with $i = 1, 2, 3, ..., h_j$, and h_j being the number of hours between the date the user-story was put in the state *"In Progress"* s_i and the date the user-story was put in the state *"Done"* f_i. At each hour i of this duration, the number of user-stories st_j in the state *"In Progress"* and assigned to the developer responsible for the user-story u_j is calculated and the effort eph_i is calculated dividing 1 by the number of user-stories st_j, representing the effort per hour spent on a user-story. In short this approach analyses the time a developer spent really working on a specific user-story.

The approach outputs a W working effort for a number M of user-stories, taking into account the N historical efforts. Based on such forecast, and knowledge about the availability of the developers(full time, part-time, etc.) for the project, the users may easily arrive at a delivery date forecast.

In short, this approach analyses the time a developer spent working on a specific user-story. It is assumed that, when a developer is assigned to multiple user-stories, he splits his time equally between those user-stories. Due to lack of further information, we did not take into account non-working days (holidays, etc.).

4 IMPLEMENTATION AND OUTPUTS

The two types of forecasting discussed in Section 3 were implemented as a python script[2]. This section discusses the type of outputs and charts that are produced by this tool to show the results of the forecast. Two types of outputs are supported, for the forecast of a single user-story and for the forecast of a set of user-stories. The latter is supported by charts.

When forecasting for a single user-story, the output produced is a simple print of the values forecasted. The model prints the numeric identifier of the project, the point forecast (median) and the confidence interval forecast.

Figure 1 shows the output of the forecast of an example of one single user-story when using takt time to forecast. On Figure 2 is outputted the effort forecast for an example of one single user-story.

```
Project ID: 177
Point Forecast: 2:36:27
Interval Forecast: [0:06:19 -- 1 day, 21:32:09]
```

Figure 1: Example of an output of the takt time approach for a single user-story

```
Project ID:798
Point Forecast: 4:12:00
Interval Forecast: [1:20:00 -- 8:00:00]
```

Figure 2: Example of an output of the effort approach for a single user-story

These outputs are very simple and due to their use on single story forecasts, they do not convey a lot of information to a user.

When forecasting for a set of user-stories, the amount of information given to the user about the forecast can be more specific and allow a better perspective of the forecast for isolated user-stories in the middle of the set of user-stories. To produce an output with those characteristics, instead of simply printing the forecast of the full amount of user-stories, a chart is built where the forecast of each number of user-stories is represented, that is, for each number of user-stories until the final forecast for M user-stories, a forecast is produced and represented in this chart output.

Figure 3 is an example of the output produced when using the takt time approach to forecast for a set of user-stories.

In this approach, the model produces a chart where the historical data is represented (green line) and the forecast for the M number of user-stories to forecast, starting after the last historical user-story, is also drawn (blue line). The forecast includes the median forecast and the confidence interval forecast. It is considered a pessimist forecast, the forecast made using the highest values inside the 95% confidence interval. An optimist is a forecast made using the lowest values inside the same 95% confidence interval. In this chart are

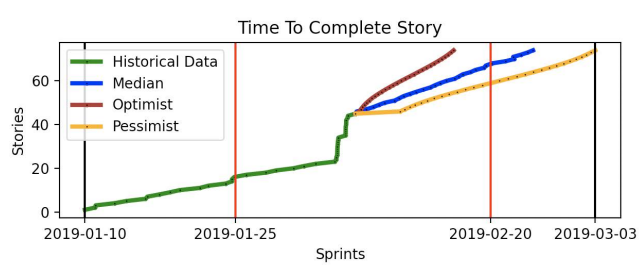

Figure 3: Example of an output using the takt time approach for a set of user-stories

[1]The *effort* was calculated by counting the number of user-stories that were assigned to a developer at the time that a specific user-story was put in the state *"In Progress"* until the time the user-story was put in the state *"Done"*. This was done because there was no explicit information about the effort per user-story on the dataset, but only the developer assigned and the dates when user-story states were changed, as mentioned above.

[2]The source-code is freely available at https://github.com/pmiranda07/Promessa

also represented the completion date of the first user-story of the historical data and the completion date of the last user-story of the pessimist forecast. In between these extremes, the project finishing dates of the sprints that occurred and the ones that are planned to occur are also represented to give a better perspective of the number of user-stories that were completed and the ones the model forecast to be completed in each sprint.

When using the effort approach to forecast for a set of user-stories the model also produces a chart where the number M of user-stories to forecast are represented with the number of hours for completing that M user-stories. The chart includes the median forecast and the confidence interval forecast. Figure 4 is an example of the chart produced when using effort to forecast a set of user-stories. In this chart, the historical data is not represented, contrary to what happens in the takt time chart, since in effort we do not consider the concept of continuity. That is, this chart represents the effort per number of user-stories, while in the takt time the chart, besides showing us the takt time between user-stories, also presents us the dates of sprints and delivery of the number of user-stories, using the concept of continuity to represent these dates.

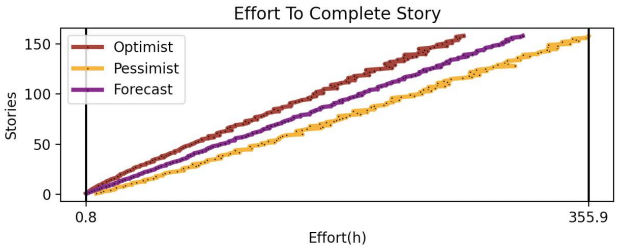

Figure 4: Example of an output of the effort approach for a set of user-stories

5 EVALUATION

In this section, we evaluate the delivery and effort forecasting approaches described in the previous section through a few experiments. For each approach, we discuss positive and negative points, evaluate which renders better results for different contexts and problems, and analyze the forecasting accuracy.

Our goal is to study more thoroughly the use of the Monte Carlo method for forecasting the delivery of software, namely in what concerns its accuracy, something that we can't find in the most closely-related works [16, 27]. To do this, we compare the method's accuracy against user's estimations when possible. Such analysis is something that we could also not yet find in current literature and that we hope will support the discussion about the relative merits of developer-provided estimates versus data-driven forecasts.

5.1 Dataset

The dataset used to validate the implementation was extracted from a self-hosted instance of Jira[3] used by Fraunhofer Portugal. The implementation used for our experiments was developed with the Jira structure in mind, what means that it has the potential to be applied in other contexts where Jira is used. Such structure is presented in Figure 5.

The information was extracted using the JIRA REST API, using the endpoints to retrieve the projects and all issues available in the database. The information was then converted to customized JSON objects that would make the information readable (there are several customized fields in JIRA). Finally, due to legal reasons, all projects and collaborators' names were properly anonymized.

The dataset contains 71 projects, each one with several user-stories. It has around 120 users who take actions on the projects, more than 12000 user-stories divided by the 71 projects, around 135000 changes made to user-stories, reported in a changelog, and almost 150 sprints. The dataset contains data about user-stories and projects from 2013 to 2020.

On this evaluation process, of the several existing projects, was given preference to the projects developed in the later years (2018, 2019, 2020), since those are more representative of the present days. Figure 5 shows the most important part of the structure of the dataset, for our purposes. Information not used for forecasting purposes is not represented in this diagram.

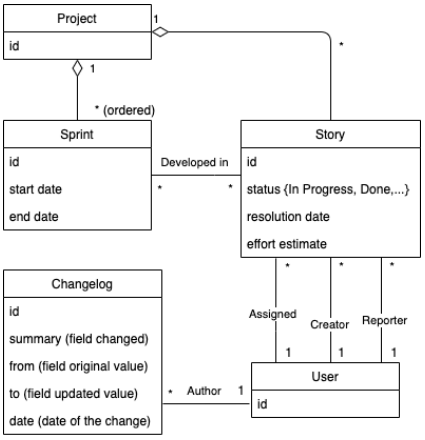

Figure 5: Dataset structure as a UML class diagram

5.2 Evaluation of the Delivery Forecasting Approach

The first experiment consisted of forecasting the completion date for a set of user-stories, based on the elapsed time between the completion dates of past user-stories (takt time). The forecasted delivery dates are compared with the values that actually occurred[4]

In the experiment, we take 50% of the user-stories as historical data (training data), and use the subsequent 50% for validation (i.e., for comparing predicted delivery dates with real values).

Figure 6 presents an example of an output obtained by running this experiment for an individual project in the dataset. The green

[3]More about Jira can be found at https://www.atlassian.com/software/jira

[4]The implementation developed to run this experiment is publicly available and described at https://github.com/pmiranda07/Promessa/blob/master/README.md#takt-time-forecast-for-a-set-of-stories

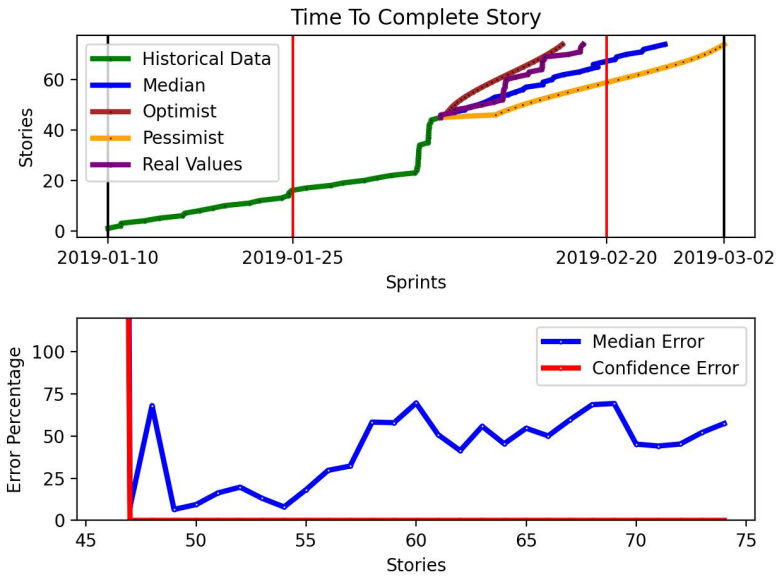

Figure 6: Example output in the delivery forecasting experiment

line represents the completion dates of the user-stories used as historical data (training data). The blue line represents the forecasted completion dates for the subsequent user-stories, with 50/50 probability (median of values generated by the Monte Carlo simulation for each user-story). The brown and yellow lines represent, respectively, the optimist and pessimist forecasts for each user-story, i.e., the lower and upper bounds of a prediction interval with 95% confidence (containing 95% of the values generated by the Monte Carlo simulation for each user-story). The sprints of the project are represented by the red lines, being the black lines the completion date of the first user-story and the pessimist forecast of the last user-story. The purple line represents the real completion dates of the user-stories to forecast. In the second chart we can see the error percentage (MRE) of the forecast for each user-story predicted, as well as the confidence error, that is, the percentage error of the real values in relation to the confidence interval.

We repeated this experiment for 10 different projects in the dataset. In each project, we picked 50% of the user-stories as historical data (or training data), and predicted the completion date of the remaining set of user-stories. The results are summarized in Table 2. For each project, the table shows the number of historical data points used to make the forecast, the forecast, the 95% confidence interval, the real delivery date, and the MMRE. The forecast, confidence interval, and real delivery are represented in days remaining up to the final delivery date.

Hist	Forecast	Conf. Interval	Real Delivery	MMRE
116	48.8	[39.4 - 56.4]	57.4	39%
49	27.5	[22.8 - 38.4]	22.4	29%
221	213.5	[193.7 - 230.1]	204.4	20%
86	35.4	[27.8 - 42.5]	33.1	27%
129	43.5	[33.1 - 51.1]	48.9	37%
101	34.5	[26.5 - 43.5]	37.9	26%
141	96.8	[82.4 - 110.4]	89.5	43%
8	7.2	[3.5 - 10.4]	6.5	21%
77	32.5	[25.1 - 39.8]	29.6	35%
114	88.1	[76.7 - 99.2]	92.4	38%

Table 2: Results of delivery forecasting experiment

5.3 Impact of Historical and Forecasting Horizons on Forecasting Accuracy

The second experiment made with the takt time has the goal to find, in this context, what were the best numbers of user-stories to use as historical data and as forecast[5], for all the projects available in the dataset. In this experiment, we want to know the percentage error of the forecasts in relation to the number of user-stories to use as historical data and the number of user-stories to forecast. To aggregate the projects per historical and future number of user-stories is used the mean of relative error or percentage error.

Figure 7 shows the results of this second experiment with takt time. For every project of the dataset, the percentage error of the forecasts in relation to the number of user-stories to use as historical data and the number of user-stories to forecast is shown.

Each line represents the forecasts for a different number of historical user-stories. Using only 5 or 10 user-stories as historical data provides higher errors, so we can say that in projects with less than 15 user-stories, the model is noticeably less accurate. We

[5]The implementation developed to run this experiment is publicly available and described at https://github.com/pmiranda07/Promessa/blob/master/README.md#takt-time-historicalforecast

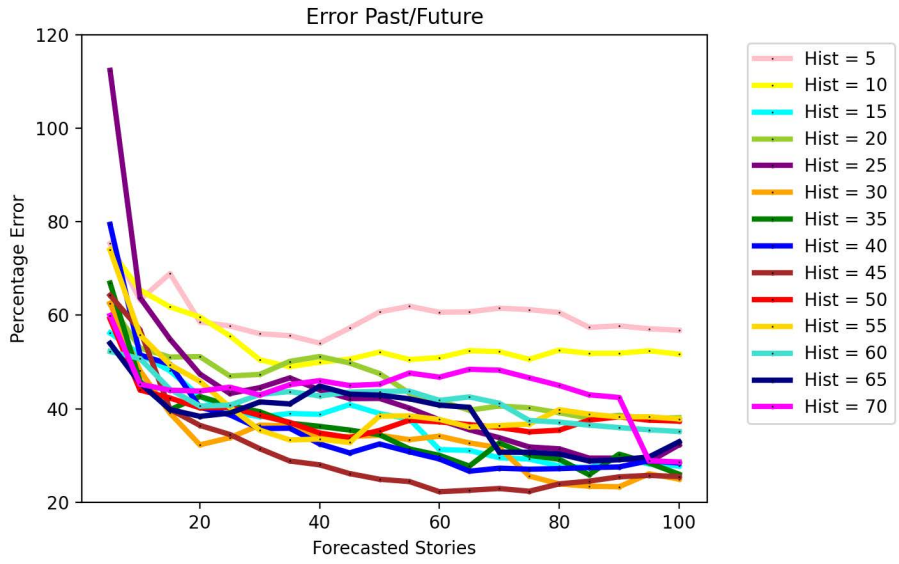

Figure 7: Forecasting errors for different amounts of user-stories used as historical data

also note that using 60, 65, or 70 user-stories as historical also especially increases the error of the forecast, probably because of more noise added into the model. Therefore, the ideal number of historical user-stories in this context is with *"Hist = 45"*. This provides better results than with other values for most of the number of user-stories to forecast, getting to its lowest point when forecasting 60 user-stories. This context—using 45 user-stories to forecast 60 user-stories into the future—is the ideal operation context for this model. The model, as stated before, presents better results when using more than 10 user-stories as historical data.

We also highlight a noticeable improvement of the error when forecasting for 20 user-stories or more. It is also to expect that, when forecasting too much into the future, the error will increase again since, the further we look into the future, more errors will accumulate. This may happen due to the accumulation of error as more user-stories need to be considered in the forecast, or to the accumulation of changes in the context of the project that imply a different delivery behavior, such as changes in the constitution of the team, or gains of experience with specific technologies.

5.4 Evaluation of the Effort Forecasting Approach

In the third experiment, we assessed the performance of the Monte Carlo method in forecasting the development effort (in hours) for a set of user-stories[6].

A difficulty that we faced in this experiment, representative of many real world situations, was the absence from the dataset of the *actual effort* of the user-stories. The only data present for each is the assigned developer and the start and finish date. *Effort estimates* were available but only for some projects.

To overcome this difficulty, we computed approximate values for the development effort spent in each user-story, based on the following simplifying assumptions: (i) developers work 8 hours per day; (ii) when a developer is allocated to multiple user-stories for a period of time, we assume that he/she splits his/her time equally by the different user-stories.

Figure 8 presents an example output produced in this experiment for an individual project in the dataset. As in previous experiments, half of the user-stories are used as historical data (training data), and the other half is used for validation purposes. More specifically, we take as historical data the approximate effort values of the first 50% of user-stories. Using the Monte Carlo simulation method, we predict effort values for the subsequent user-stories, and display the accumulated values for the median, optimistic and pessimistic effort forecasts. Approximate real values are shown in green and developer estimates are shown in blue.

We repeated this experiment for 5 projects in the dataset that had effort estimates available for all user-stories. Table 3 summarizes the results. The mean of the MMRE of these 5 simulations is 20%, while the mean of the MMRE for the users' estimates is 134%.

These results suggest that the Monte Carlo method is able to provide useful effort forecasts for a set of user-stories, even better than the estimates produced by the developers themselves. Accurate effort forecasts are important to estimate other project management variables, such as cost and schedule, needed to make informed commitments with customers.

[6]The implementation developed to run this experiment is publicly available and described at https://github.com/pmiranda07/Promessa/blob/master/README.md#effort-forecast-for-a-set-of-stories

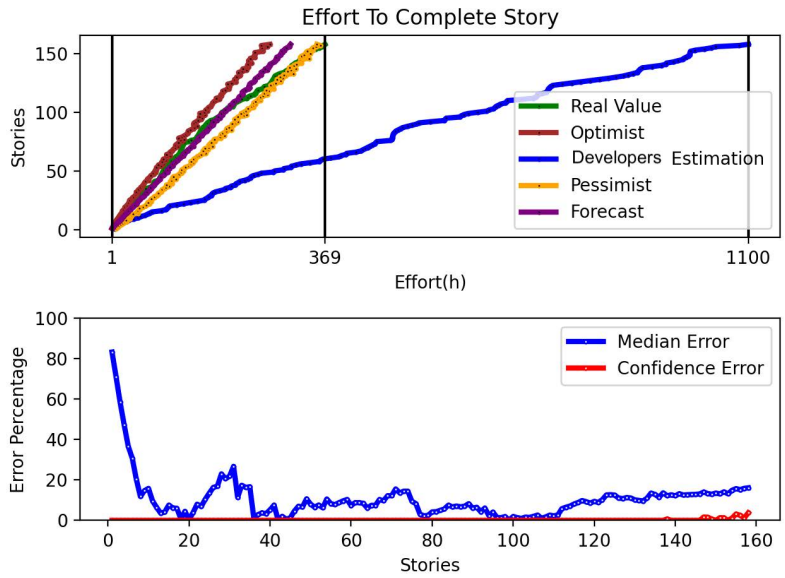

Figure 8: Example output of the effort forecasting experiment

Hist	Forecast	95% Prediction Interval	Real Effort (approx.)	User Estimate	MMRE of Forecast	MMRE of User Estimate
101	306h	[287h - 369h]	378h	1100h	20%	191%
16	75h	[55h - 94h]	68h	129h	11%	89%
12	95h	[83h - 103h]	59h	87h	31%	48%
17	81h	[70h - 93h]	87h	237h	12%	172%
14	69h	[54h - 85h]	65h	177h	27%	172%

Table 3: Results of effort forecasting experiment

5.5 Limitations

In this section, we identify possible flaws in our methodology and reasons why the results might not generalize, and explain our efforts to mitigate such possibilities.

The evaluation of the method using only this one dataset may put into question its accuracy when applied to other contexts. Although the dataset has a reasonable number of projects, we would like to run more experiments on even more projects, with different contexts, different domains, that use different development methodologies and that come from a company environment, in order to increase the confidence in the results obtained and reported here.

It would find it interesting to study the influence of additional variables in the quality of the outputs of the algorithm. We hypothesize that variables like the *user-stories size*, the developers' expertise, or the type of the project, may also play an important in delivery. Unfortunately, in the dataset used these variable were not available, and were not possible to obtain by other means.

Another issue that could threaten the legitimacy of the results are the assumptions made in the effort forecasting approach. It was assumed that the developer works the same amount of time per hour on all its assigned user-stories, what may not not represent the reality, because what really happens is that a developer probably works some hours on one user-story and then shifts to another. Assuming that only the assigned developer works on a specific user-story could also be wrong, since sometimes other developers of the team responsible for the project where the user-story is inserted in can also work on the user-story, increasing the effort that will not be taking into account. It is also assumed that a developer only works 8 hours per day, another assumption that does not necessarily happen all the time, since sometimes developers do extra hours, increasing the hours per day. Since the real values of effort are also calculated using these assumptions since no record of effort is provided by the dataset, this could explain the overestimates made by the users. All these assumptions are made as an alternative to work with the information in the dataset, due to the lack of information about the real effort and lack of information about which developers worked in the user-story, and because of this could represent threats to the legitimacy of the results presented.

The use of MMRE as a accuracy indicator may also be seen as a concern and a possible threat to the validity of our results. Despite being widely used for this purpose, MMRE suffers from some drawbacks; it is an asymmetric indicator, since underestimates cannot exceed 100% whereas overestimates are unbounded, and it

is a dispersion measure. These characteristics make it problematic when dealing with error compensation [21].

The results shown in Figure 8 suggest that, when we look too much into the future (in this case, after around 100 user-stories), effort forecasts start to get less accurate. However, further experiments are needed to confirm this observation (which matches common intuition).

In this paper, we did not made a comparison of the errors shown by the Monte Carlo method in the experiments done against other methods experiments described in other papers available. This is a valuable discussion to make, as it can give us practical evidence of which method provides better accuracy and smaller errors.

6 CONCLUSION

The document presented, analyses, and reports important notions about this field, presents a report of some of the most used metrics, both for measurement of input/output and for validation. It analyses some of the most used methods and reports their major benefits, limitations, and make a comparison between them, highlighting the most promising methods.

After selecting the Monte Carlo method, due to its simple applicability in agile context, its understandability and its granularity, a forecasting model was implemented. Having 2 different delivery forecasting approaches implemented, in order to evaluate, we recurred to a dataset to build some experiments. With this evaluation of each approach, we discuss some of the negative and positive points of each one of those approaches and analyzed their accuracy.

In the future, an implementation of a machine learning method could also be relevant, since this type of methods also has a lot of positive points that can help to provide more accurate results, as the possibility of discovering existing patterns between projects, the avoidance of overfitting to data, that is, the ease to adapt to new situations, something that does not happen with the Monte Carlo method, and the generalization capability.

An advantage of this study is that it can be used as a baseline to evaluate other methods and compare the results. It can be used, particularly, as a baseline for a study using Machine Learning methods. Takt Time and Effort are, based on the findings, two good delivery forecasting data features to use on this Machine Learning implementation.

ACKNOWLEDGMENTS

This article is a result of the project PROMESSA - NORTE-01-0247-FEDER-039887, supported by Norte Portugal Regional Operational Programme (NORTE 2020), under the PORTUGAL 2020 Partnership Agreement, through the European Regional Development Fund (ERDF).

REFERENCES

[1] Mohamed Abouelela and Luigi Benedicenti. 2010. Bayesian network based XP Process Modelling. *International Journal of Software Engineering & Applications* 1 (July 2010). https://doi.org/10.5121/ijsea.2010.1301

[2] Pekka Abrahamsson, Raimund Moser, Witold Pedrycz, Alberto Sillitti, and Giancarlo Succi. 2007. Effort Prediction in Iterative Software Development Processes – Incremental Versus Global Prediction Models. In *First International Symposium on Empirical Software Engineering and Measurement, ESEM 2007*. IEEE, Madrid, Spain, 344–353. https://doi.org/10.1109/ESEM.2007.16

[3] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Molina, Richard Benjamins, et al. 2020. Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion* 58 (2020), 82–115.

[4] Thomas Betts. 2019. Web-Based Monte Carlo Simulation for Agile Estimation. Available at https://mobilemonitoringsolutions.com/web-based-monte-carlo-simulation-for-agile-estimation/.

[5] Roheet Bhatnagar, Vandana Bhattacharjee, and Mrinal Ghose. 2010. Software Development Effort Estimation – Neural Network Vs. Regression Modeling Approach. *International Journal of Engineering Science and Technology* 2 (07 2010).

[6] Pawel Brodzinski. 2015. Estimation and Forecasting. Available at https://www.agilealliance.org/estimation-and-forecasting//.

[7] Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Machine Learning* 20 (1995), 273 – 297. https://doi.org/10.1007/BF00994018

[8] Wei Du, Luiz Capretz, and Danny Ho. 2010. A Neuro-Fuzzy Model with SEER-SEM for Software Effort Estimation. In *25th International Forum on COCOMO and Systems/Software Cost Modeling*. Los Angeles, CA, USA, 7.

[9] Richard E. Fairley. 1992. Recent Advances in Software Estimation Techniques. In *ICSE '92: Proceedings of the 14th international conference on Software engineering*, ACM (Ed.). ACM, Melbourne, Australia, 382–391. https://doi.org/10.1145/143062.143155

[10] Adrian Fittolani. 2015. Agile Project Forecasting – The Monte Carlo Method. Scrumage. Available at http://scrumage.com/blog/2015/09/agile-project-forecasting-the-monte-carlo-method/.

[11] James Grenning. 2002. Planning Poker.

[12] Ali Idri, Alain Abran, and Taghi Khoshgoftaar. 2002. Estimating software project effort by analogy based on linguistic values. In *Proceedings Eighth IEEE Symposium on Software Metrics*. IEEE, Ottawa, Canada, 21–30. https://doi.org/10.1109/METRIC.2002.1011322

[13] Galorath Incorporated. 2011. SEER-SEM Product Brief. Available at https://galorath.com/wp-content/uploads/2020/09/SEER-for-Software-Datasheet-092920.pdf.

[14] Liz Keogh. 2015. The Estimates in #NoEstimates. Available at https://lizkeogh.com/2015/05/01/the-estimates-in-noestimates/.

[15] Ekrem Kocaguneli, Ayse Tosun, and Ayse Bener. 2010. AI-Based Models for Software Effort Estimation. In *36th EUROMICRO Conference on Software Engineering and Advanced Applications*, IEEE (Ed.). IEEE, 323–326. https://doi.org/10.1109/SEAA.2010.19

[16] Troy Magennis. 2012. Managing Software Development Risk using Modeling and Monte Carlo Simulation. In *Proceedings of the Lean Software & Systems Conference 2012*. Boston, USA. https://doi.org/10.13140/2.1.3708.1927

[17] Dan North. 2018. Monte Python Simulation: Misunderstanding Monte Carlo. Available at https://dannorth.net/2018/09/04/monte-python-simulation/.

[18] Parag Pendharkar, Girish Subramanian, and James Rodger. 2005. A Probabilistic Model for Predicting Software Development Effort. *Software Engineering, IEEE Transactions on* 31 (08 2005), 615– 624. https://doi.org/10.1109/TSE.2005.75

[19] Shashank Mouli Satapathy, Aditi Panda, and Santanu Rath. 2014. Story Point Approach based Agile Software Effort Estimation using Various SVR Kernel Methods. In *The 26th International Conference on Software Engineering and Knowledge Engineering*, Vol. 2014. Knowledge Systems Institute Graduate School, Vancouver, Canada.

[20] Christopher Schofield. 1998. *An Empirical Investigation into Software Effort Estimation by Analogy*. Ph.D. Dissertation. Bournemouth University.

[21] Martin Shepperd, Michelle Cartwright, and Gada Kadoda. 2000. On Building Prediction Systems for Software Engineers. *Empirical Software Engineering* 5 (2000), 175–182.

[22] Dr. Amit Sinhal and Bhupendra Verma. 2013. Software Development Effort Estimation: A Review. *International Journal of Advanced Research in Computer Science and Software Engineering* 3 (06 2013), 1120–1135.

[23] Bradley R Staats, David James Brunner, and David M Upton. 2011. Lean principles, learning, and knowledge work: Evidence from a software services provider. *Journal of operations management* 29, 5 (2011), 376–390.

[24] Andrew Stellman and Jennifer Greene. 2005. Applied Software Project Management – Software Project Planning Practices. Available at https://www.stellman-greene.com/about/applied-software-project-management/applied-software-project-management-software-project-planning-practices/.

[25] D. Toka and Oktay Turetken. 2013. Accuracy of Contemporary Parametric Software Estimation Models: A Comparative Analysis. In *Proceedings - 39th Euromicro Conference Series on Software Engineering and Advanced Applications, SEAA 2013*. IEEE, Santander, Spain, 313–316. https://doi.org/10.1109/SEAA.2013.49

[26] Sharma Vishal and Harsh Verma. 2010. Optimized Fuzzy Logic Based Framework for Effort Estimation in Software Development. *International Journal of Computer Science Issues* 7 (04 2010).

[27] Juanjuan Zang. 2008. Agile Estimation with Monte Carlo Simulation. In *Agile Processes in Software Engineering and Extreme Programming*, Pekka Abrahamsson, Richard Baskerville, Kieran Conboy, Brian Fitzgerald, Lorraine Morgan, and Xiaofeng Wang (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 172–179.